

Question 1(iii): Coordinate Search

Perform two steps of coordinate search with learning rate $\alpha = 0.15$, starting at $x_0 = (2, 2)^T$.

The function is: $f(x, y) = 4x^2 - 2xy + y^2 - 3x + 2y + 7$

Method (Section 3.1.1 of lecture notes — ZERO-ORDER method)

Coordinate search makes **no use of derivatives**. At each sub-step, for coordinate m , we evaluate two candidate points:

$$w^{\text{candidate}} = w \pm \alpha e_m$$

where e_m is the m -th standard basis vector. We accept whichever gives the lower function value, provided it improves on the current value. One full **step** cycles through all coordinates (x , then y).

```
In [1]: import numpy as np

def f(x, y):
    """Objective function: f(x,y) = 4x^2 - 2xy + y^2 - 3x + 2y + 7"""
    return 4*x**2 - 2*x*y + y**2 - 3*x + 2*y + 7

alpha = 0.15
x, y = 2.0, 2.0
n_steps = 2

print(f"Starting point: ({x:.3f}, {y:.3f}), f = {f(x, y):.3f}")
print()

for step in range(1, n_steps + 1):
    print(f"=== Step {step} ===")

    f_current = f(x, y)
    f_plus = f(x + alpha, y)
    f_minus = f(x - alpha, y)

    print(f"  Coordinate x  (current x={x:.3f}, y={y:.3f}, f={f_current:.3f}):")
    print(f"    f({x+alpha:.3f}, {y:.3f}) = {f_plus:.3f}    [x + alpha]")
    print(f"    f({x-alpha:.3f}, {y:.3f}) = {f_minus:.3f}    [x - alpha]")

    best_f_x = min(f_plus, f_minus)
    if best_f_x < f_current:
        x = (x + alpha) if (f_plus <= f_minus) else (x - alpha)
        print(f"    -> Accept: x updated to {x:.3f}, f = {f(x,y):.3f}")
    else:
        print(f"    -> No improvement; x unchanged at {x:.3f}")

    f_current = f(x, y)
```

```

f_plus    = f(x, y + alpha)
f_minus   = f(x, y - alpha)

print(f"  Coordinate y  (current x={x:.3f}, y={y:.3f}, f={f_current:.3f}):")
print(f"    f({x:.3f}, {y+alpha:.3f}) = {f_plus:.3f}    [y + alpha]")
print(f"    f({x:.3f}, {y-alpha:.3f}) = {f_minus:.3f}    [y - alpha]")

best_f_y = min(f_plus, f_minus)
if best_f_y < f_current:
    y = (y + alpha) if (f_plus <= f_minus) else (y - alpha)
    print(f"    -> Accept: y updated to {y:.3f}, f = {f(x,y):.3f}")
else:
    print(f"    -> No improvement; y unchanged at {y:.3f}")

print(f"  End of step {step}: ({x:.3f}, {y:.3f}), f = {f(x, y):.3f}")
print()

print(f"Final point after {n_steps} steps: ({x:.3f}, {y:.3f})")
print(f"f(final) = {f(x, y):.3f}")

```

Starting point: (2.000, 2.000), f = 17.000

=== Step 1 ===

```

Coordinate x  (current x=2.000, y=2.000, f=17.000):
  f(2.150, 2.000) = 18.440    [x + alpha]
  f(1.850, 2.000) = 15.740    [x - alpha]
  -> Accept: x updated to 1.850, f = 15.740
Coordinate y  (current x=1.850, y=2.000, f=15.740):
  f(1.850, 2.150) = 16.108    [y + alpha]
  f(1.850, 1.850) = 15.418    [y - alpha]
  -> Accept: y updated to 1.850, f = 15.418
End of step 1: (1.850, 1.850), f = 15.418

```

=== Step 2 ===

```

Coordinate x  (current x=1.850, y=1.850, f=15.418):
  f(2.000, 1.850) = 16.723    [x + alpha]
  f(1.700, 1.850) = 14.293    [x - alpha]
  -> Accept: x updated to 1.700, f = 14.293
Coordinate y  (current x=1.700, y=1.850, f=14.293):
  f(1.700, 2.000) = 14.660    [y + alpha]
  f(1.700, 1.700) = 13.970    [y - alpha]
  -> Accept: y updated to 1.700, f = 13.970
End of step 2: (1.700, 1.700), f = 13.970

```

Final point after 2 steps: (1.700, 1.700)

f(final) = 13.970

Question 2(ii): Linear Regression Statistics

Compute $\bar{x}$, $\bar{y}$, and the sample correlation coefficient r_{xy} using equation (3.4.5) from the lecture notes.

In [2]: `import numpy as np`

```
x_data = np.array([1.73, 1.87, 1.90, 2.66, 3.43, 4.22])
y_data = np.array([4.61, 4.98, 4.39, 6.59, 7.62, 9.65])
n = len(x_data)

x_bar = np.mean(x_data)
y_bar = np.mean(y_data)

S_xx = np.sum((x_data - x_bar)**2)
S_yy = np.sum((y_data - y_bar)**2)
S_xy = np.sum((x_data - x_bar) * (y_data - y_bar))

r_xy = S_xy / np.sqrt(S_xx * S_yy)

print(f"Sample mean of x:  $\bar{x}$  = {x_bar:.4f}")
print(f"Sample mean of y:  $\bar{y}$  = {y_bar:.4f}")
print(f"Pearson correlation coefficient:  $r_{xy}$  = {r_xy:.4f}")
```

Sample mean of x: $\bar{x}$ = 2.6350

Sample mean of y: $\bar{y}$ = 6.3067

Pearson correlation coefficient: r_{xy} = 0.9903